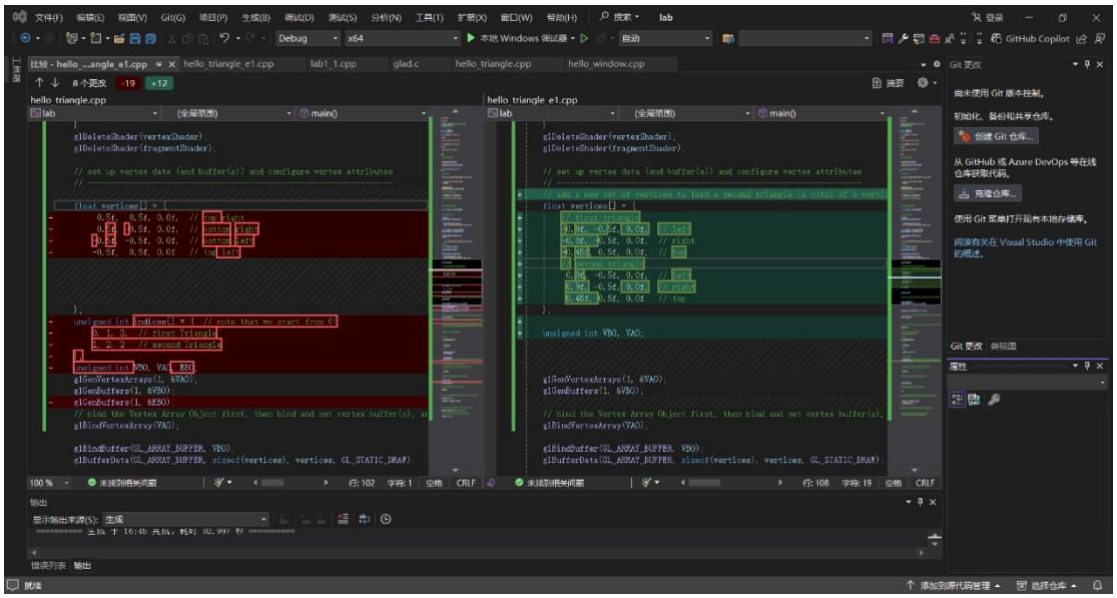
计算机图形学实验一——opengl 基础概念理解与使用：着色器

1. **对于 learnopengl 教程中的参考代码，可以通过 visual studio 的代码对比功能查看，便于理解关键代码，并反过来对有疑问处细看教程。**
   https://learn.microsoft.com/zh-cn/visualstudio/ide/compare-with?view=vs-2022

对于练习 1，主要有三处修改：增加 vertices 中的顶点（含重复），删去 indices 及相关联的 EBO，调用 glDrawArrays 而非 glDrawElements。

同样是绘制两个相连三角形，我们可以发现，indices 是更简便的用法，不但减少了存储空间，还容易从代码直观看到三角形的关联（两个三角形间至少有两个顶点索引重合为相连三角形）。

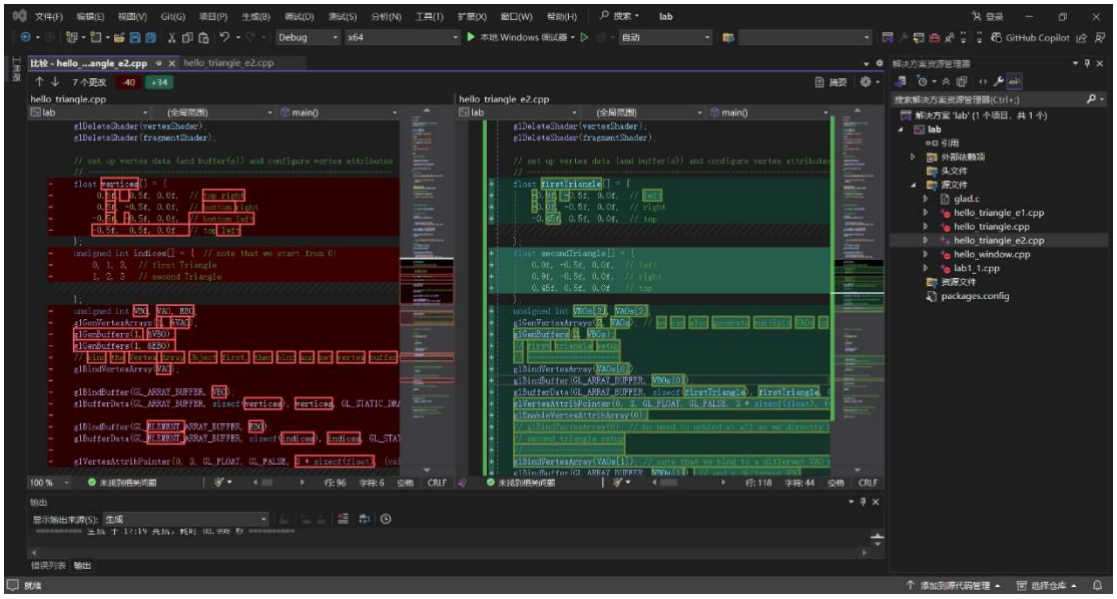
同时，我们也通过对比，看到 glDrawElements 的用法需要靠 indices 数组与 EBO 缓冲来实现，通过 glBindBuffer 与 glBufferData 来绑定绘制内容。



对于练习 2, 主要有三处修改: 增加三角形顶点数组 vertices 及相关联的 VAOs 和 VBOs, 删去 indices 及相关联的 EBO，调用多次 glDrawArrays。

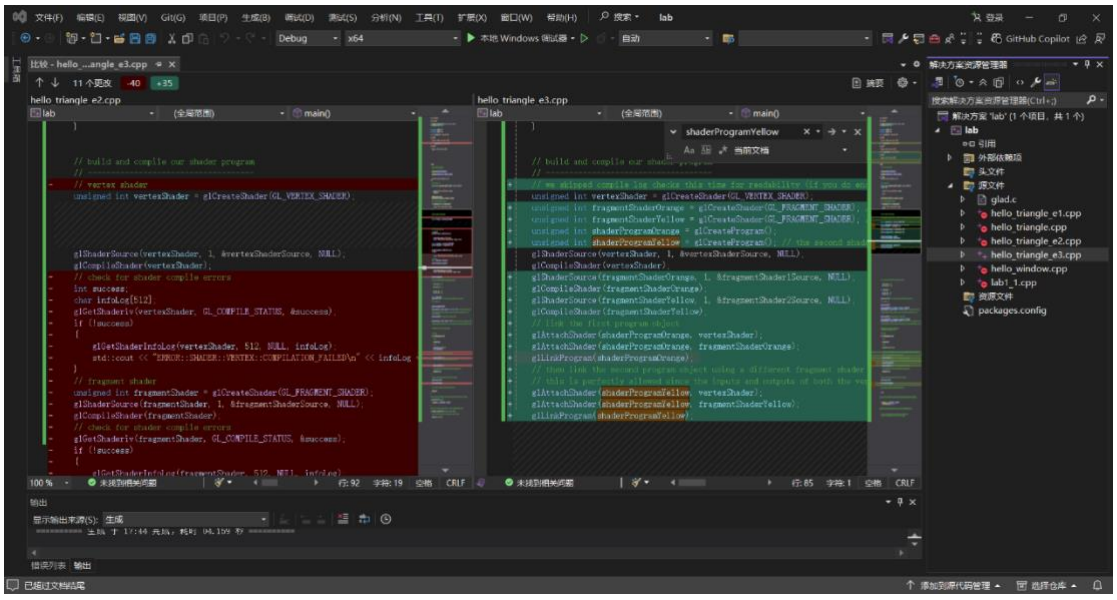
练习 2 中绘制的图形与“hello_triangle.cpp”中的不一致，我们可以从代码中 vertices 的改变对比，可以发现主要更改的是第一维，即左右方向；通过分析与修改，可以得知，vertices 中修改三个维度 x、y、z 的坐标值分别对绘制的影响。其中在无颜色无纹理的渲染中，不能体现出来修改 z 坐标的变化。

对于 VAOs、VBOs 的绑定，可以发现流程完全一致，只是多复制了一遍，主要需要注意数组指针与元素引用的区别；其中由于未使用 indices，此处使用 glDrawArrays 绘制，多次绘制的结果会累加。



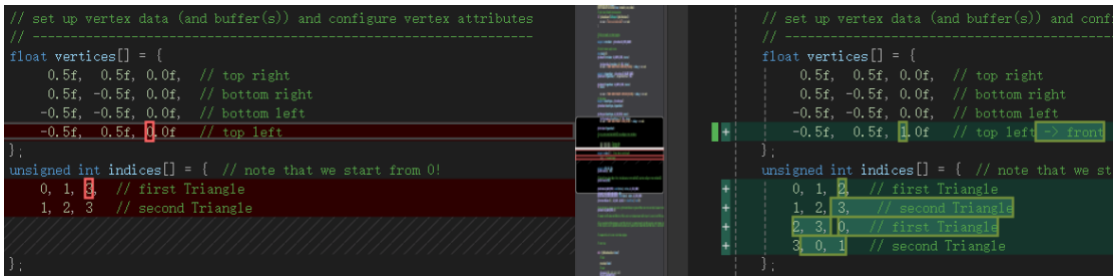
对于练习 3，与练习 2 对比，主要有三处修改：增加片段着色器，将两个片段着色器分别与顶点着色器链接为着色器程序，在绘制中用 glUseProgram 调用不同着色器。

在对比中，我们更容易理解着色器使用的流程，顶点着色器与片段着色器的关联（共同组成着色器程序，不同顶点着色器与片段着色器可以分别组合），理解链接流程等。



2. 在“hello_triangle.cpp”的基础上，渲染自定义立方体

首先，简单地将 vertices 中第四个顶点 z 坐标设为 1.0，并更新 indices 为四个三角面（注意不要重复），得到简单四面体（非正四面体）

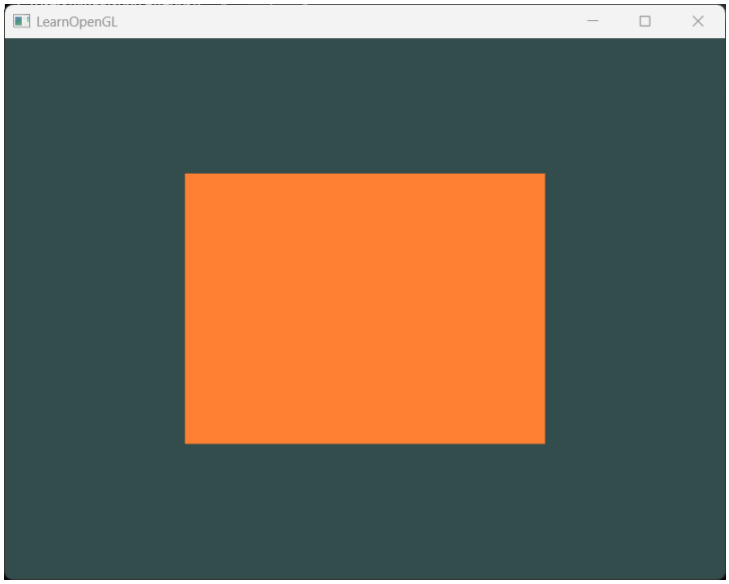



```
float vertices[] = {
     0.5f,  0.5f, 0.0f,  // top right
     0.5f, -0.5f, 0.0f,  // bottom right
    -0.5f, -0.5f, 0.0f,  // bottom left
    -0.5f,  0.5f, 1.0f   // top left -> front
};
unsigned int indices[] = {  // note that we start from 0!
    0, 1, 2,  // first Triangle
    1, 2, 3,   // second Triangle
    2, 3, 0,  // first Triangle
    3, 0, 1   // second Triangle
};
```

与更新 indices 数组对应的，glDrawElements 中的顶点数量也需要随之修改为 3*4

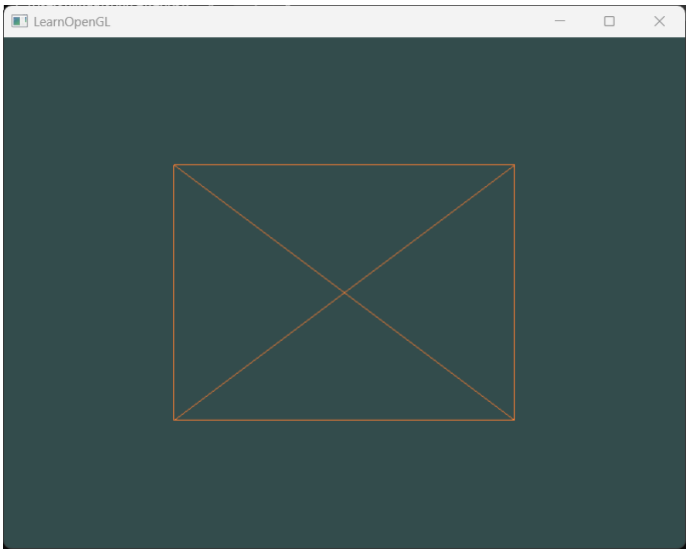
```
glDrawElements(GL_TRIANGLES, 12, GL_UNSIGNED_INT, 0);
```

直接运行，可以得到平面上显示的立方体（由于未设置颜色旋转等，仍为平面矩形）



绘制前设置线框模式，查看立体效果

```
glPolygonMode(GL_FRONT_AND_BACK, GL_LINE);
```

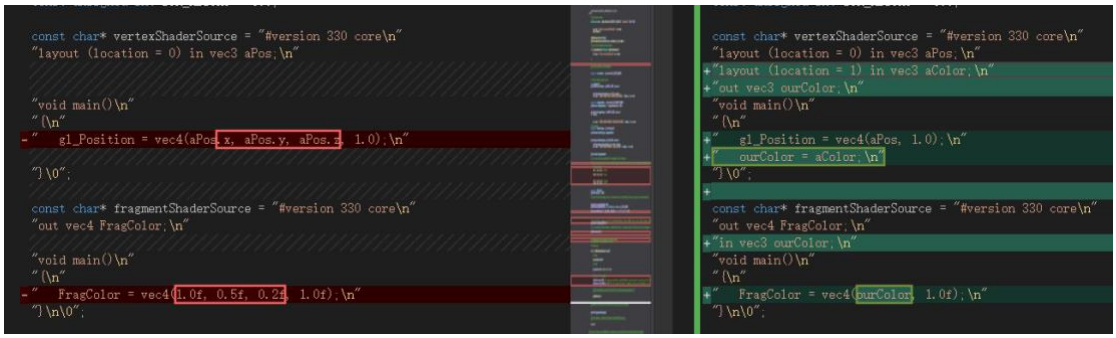


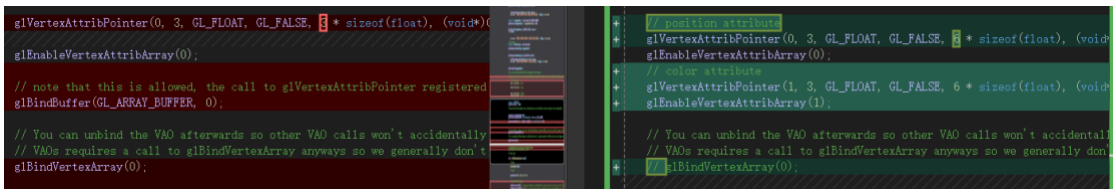
3. 阅读着色器一节，运行示例代码"shaders_interpolation.cpp"

https://learnopengl.com/code_viewer_gh.php?code=src/1.getting_started/3.2.shaders_interpolation/shaders_interpolation.cpp

由于"shaders_interpolation.cpp"中无 indices 数组，我们将其与练习 1 参考代码对比，可知设置颜色主要在以下三处进行了修改：修改 vertices 数组，增加颜色属性；增加着色器中的颜色输入源码内容；glVertexAttribPointer 中属性的修改。

重新回到教程，glVertexAttribPointer 的第一个参数与着色器源码中的"layout (location = 0)"或"layout (location = 1)"对应，我们在着色器中设置 0 为 pos、1 为 color，在后文也需要如此；第五个参数步长、第六个参数偏移量需要与 vertices 数组的元素数目相对应。





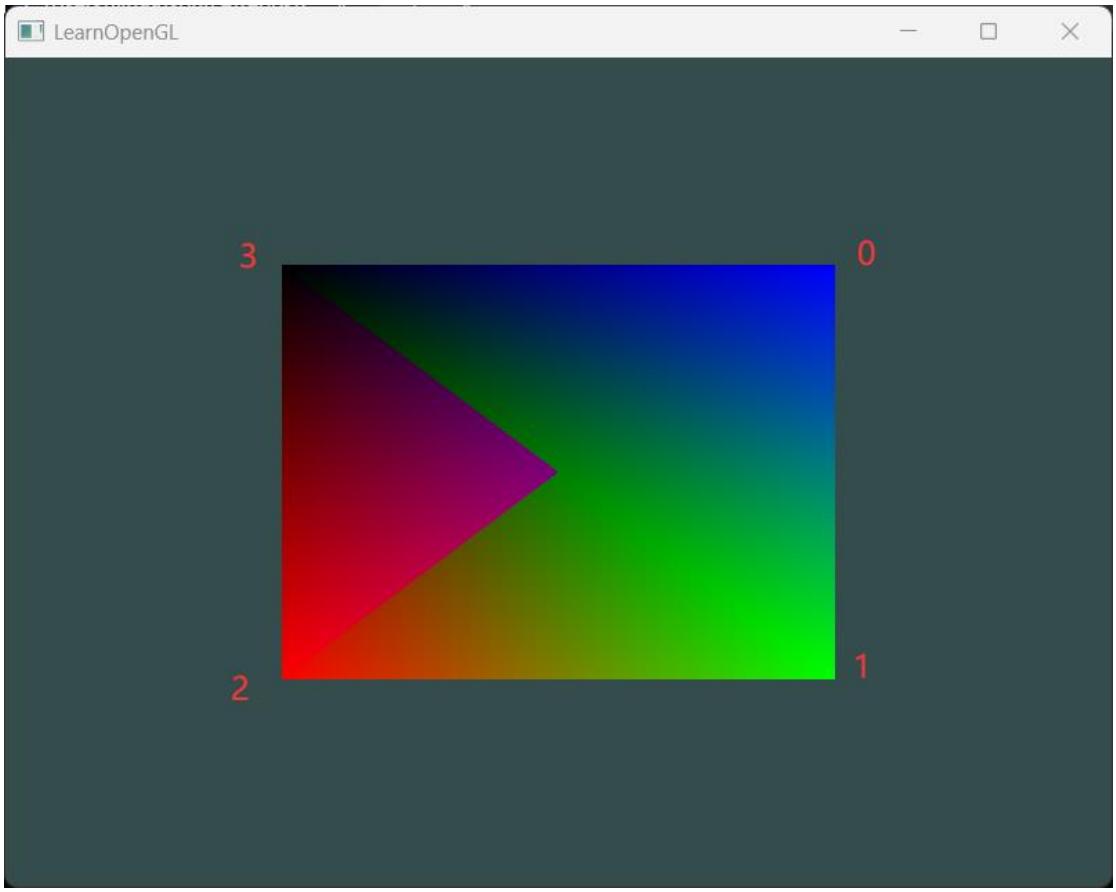
4. 在第 2 步的基础上，为自定义立方体设置颜色

根据我们对教程该节的理解，以及对示例代码的阅读，在主代码中为立方体直接设置颜色；着色器部分与 glVertexAttribPointer 部分与"shaders_interpolation.cpp"相同，直接在对

应部分添加。

```
    float vertices[] = {
         0.5f,  0.5f, 0.0f, 0.0f, 0.0f, 1.0f,  // top right
         0.5f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f, // bottom right
        -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f, // bottom left
        -0.5f,  0.5f, -1.0f, 0.0f, 0.0f, 0.0f // top left -> front
    };
    unsigned int indices[] = {  // note that we start from 0!
        0, 1, 2,  // first Triangle
        1, 2, 3,   // second Triangle
        2, 3, 0,  // first Triangle
        3, 0, 1   // second Triangle
    };
```

运行结果如下，由于顶点 3 在前，理想情况下，从正面应当能看到立方体的两个面，即(1, 2, 3)面与(3, 0, 1)面，背面为(0, 1, 2)面与(2, 3, 0)面。但此处结果并不符合，而是三角面间有重叠，这是由于我们未开启深度测试，而是直接根据 indices 中的顺序进行绘制，后绘制的面会覆盖先绘制的。

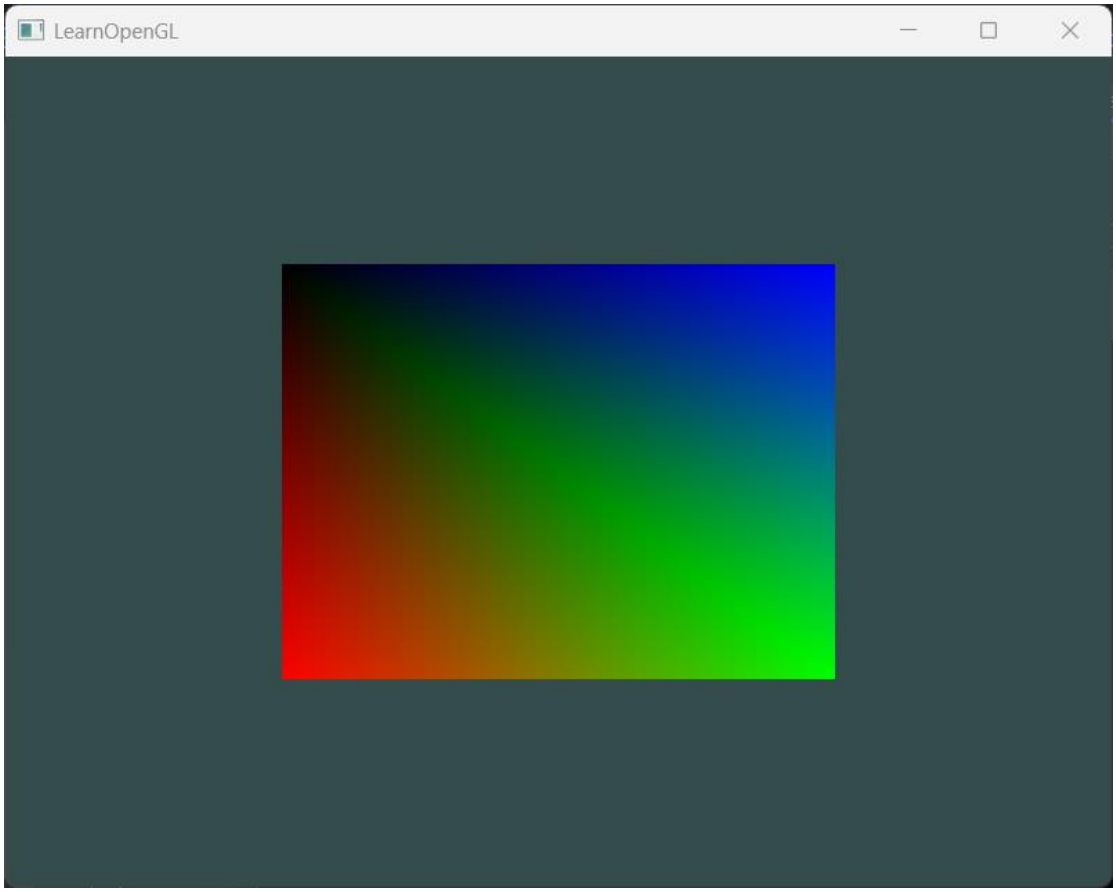


修改 indices 中三角面的顺序，使其符合视觉习惯：

```
    unsigned int indices[] = {  // note that we start from 0!
```

```
        0, 1, 2,  // first Triangle
        2, 3, 0,  // first Triangle
        3, 0, 1,   // second Triangle
        1, 2, 3,   // second Triangle
    };
```

运行结果如下，这是立方体的正面效果，更立体的展现将在下一节平移旋转中尝试。

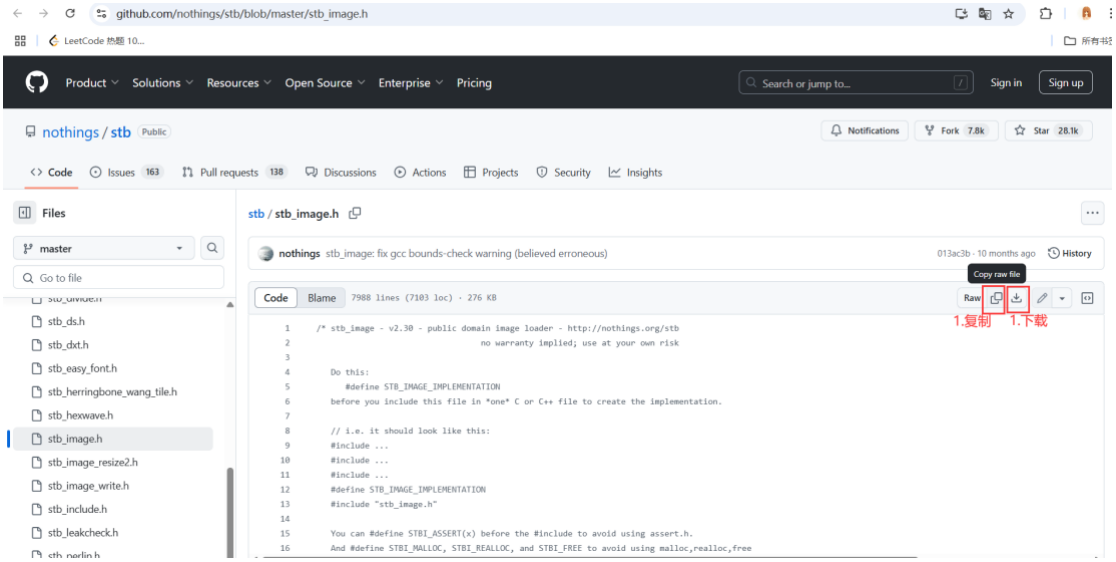


5. 阅读纹理一节，运行示例代码"shaders_interpolation.cpp"

首先下载"stb_image.h"，并添加到项目中。我们可以**选择像 glad 一样将头文件的路径添加到"附加包含目录"中**，在代码中直接使用**"#include <stb_image.h>"包含头文件**。

另一种选择，在项目中"新建项"-"stb_image.h"，复制"stb_image.h"的内容到该项中，同时在代码中修改为"#include "stb_image.h""来包含头文件。注意，此处的"#include "stb_image.h""中使用引号，而非尖括号，是指以相对路径来寻找头文件。

https://github.com/nothings/stb/blob/master/stb_image.h



同时，需要在项目中新建.cpp 文件，如“std_image_swap.cpp”，并包含教程所述代码。

https://learnopengl-cn.github.io/01%20Getting%20started/06%20Textures/#stb_imageh

教程中的纹理图像也可以从 github 下载，注意在示例代码中也需要对应修改图像文件路径为相对于项目的相对路径/本机中的绝对路径。

https://github.com/JoeyDeVries/LearnOpenGL/tree/master/resources/textures

```
unsigned char* data = stbi_load("container.jpg", &width, &height, &nrChannels, 0);
...
data = stbi_load("awesomeface.png", &width, &height, &nrChannels, 0);
```

（可选）

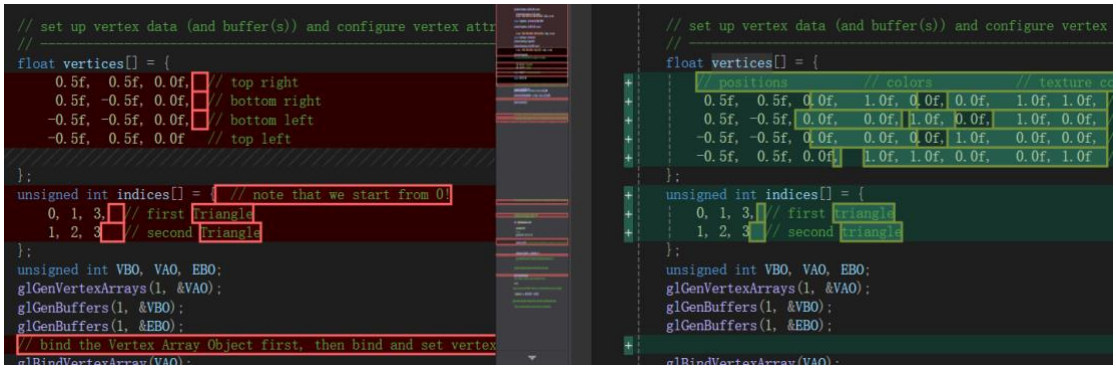
对于“shader_s.h”，我们也可以用同样的方法将其加入项目中。注意，新建项直接复制“shader_s.h”的，需要修改“#include <learnopengl/shader_s.h>”为#include “shader_s.h”。

https://learnopengl.com/code_viewer_gh.php?code=includes/learnopengl/shader_s.h

并且也需要将对应的"4.2.texture.vs", "4.2.texture.fs"添加到项目中，根据真实路径，修改对应的 Shader 初始化函数。

```
Shader ourShader("./4.2.texture.vs", "./4.2.texture.fs");
```

将”shaders_interpolation.cpp”与“hello_triangle.cpp”对比，主要区别有：vertices 数组中增加纹理属性；着色器代码；glVertexAttribPointer；纹理相关代码。



6. 在第 4 步的基础上，为自定义立方体设置纹理

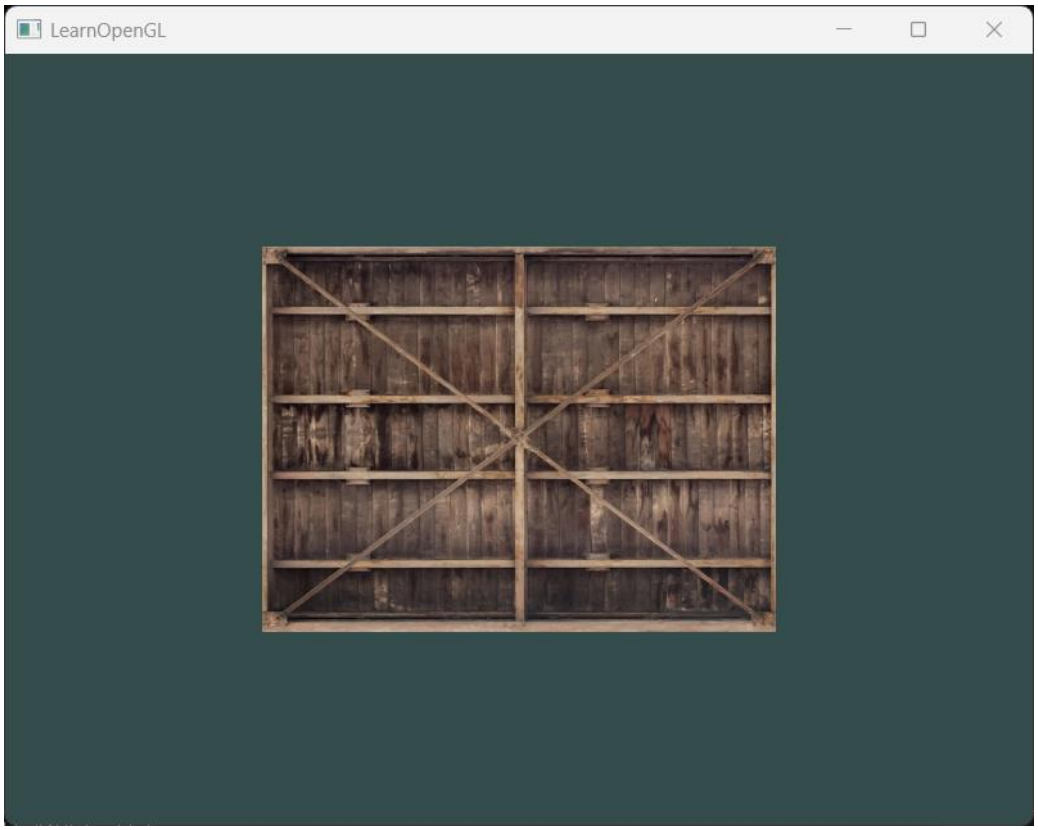
根据我们对教程该节的理解，以及对示例代码的阅读，在主代码中为立方体直接设置纹理；着色器部分与"texture.vs"，"texture.fs"相同，纹理部分与 glVertexAttribPointer 部分与“textures_combined.cpp”相同，直接在对应部分添加。

着色器内容为：

```
const char* vertexShaderSource = "#version 330 core\n"
"layout (location = 0) in vec3 aPos;\n"
"layout (location = 1) in vec3 aColor;\n"
"layout (location = 2) in vec2 aTexCoord;\n"
"out vec3 ourColor;\n"
"out vec2 TexCoord;\n"
"void main()\n"
"{\n"
"    gl_Position = vec4(aPos, 1.0);\n"
"    ourColor = aColor;\n"
"    TexCoord = vec2(aTexCoord.x, aTexCoord.y);\n"
"}\0";

const char* fragmentShaderSource = "#version 330 core\n"
"out vec4 FragColor;\n"
"in vec3 ourColor;\n"
"in vec2 TexCoord;\n"
"uniform sampler2D ourTexture;\n"
"void main()\n"
"{\n"
"    FragColor = texture(ourTexture, TexCoord);\n"
"}\n\0";
```

Vertices 数组内容为：

```
    float vertices[] = {
        0.5f,  0.5f, 0.0f, 0.0f, 0.0f, 1.0f, 1.0f, 1.0f,  // top right
        0.5f, -0.5f, 0.0f, 0.0f, 1.0f, 0.0f, 1.0f, 0.0f,// bottom right
       -0.5f, -0.5f, 0.0f, 1.0f, 0.0f, 0.0f, 0.0f, 0.0f,// bottom left
       -0.5f,  0.5f, -1.0f, 0.0f, 0.0f, 0.0f, 0.0f, 1.0f // top left -> front
    };
```

运行结果如下所示，由于在绑定纹理时直接设置四个顶点为边界，且在第 4 步时，我们只显示表面的两个三角面，在当前视角的结果与平面相同：